retail_Sliver_layer:

```
AIRBNBRETAIL.BRONZE.CUSTOMER_STAGE----------------sliver layer----------------------------------------
use schema sliver;

-- Set correct role and database
USE ROLE ACCOUNTADMIN;
USE DATABASE RETAIL;

-- Create SILVER schema if it does not exist
CREATE SCHEMA IF NOT EXISTS RETAIL.SILVER;

-- Create Silver products table
CREATE OR REPLACE TABLE RETAIL.SILVER.PRODUCTS AS
SELECT DISTINCT
    product_id,
    TRIM(product_name) AS product_name,
    TRIM(category) AS category,
    TRY_TO_NUMBER(price) AS price,
    CURRENT_TIMESTAMP() AS processed_ts
FROM RETAIL.BRONZE.PRODUCTS
WHERE product_id IS NOT NULL;

select * from RETAIL.SILVER.PRODUCTS;


CREATE OR REPLACE TABLE retail.silver.customers AS
SELECT
  customer_id,
  TRIM(customer_name) AS customer_name,
  TRIM(gender)        AS gender,
  TRY_TO_NUMBER(age)  AS age,
  TRIM(city)          AS city,
  CURRENT_TIMESTAMP() AS processed_ts
FROM retail.bronze.customers
QUALIFY ROW_NUMBER() OVER (
  PARTITION BY customer_id
  ORDER BY customer_id
) = 1;
```

```
CREATE OR REPLACE TABLE retail.silver.sales AS
SELECT
    customer_id,
    product_id,
    store_id,
    TRY_TO_NUMBER(quantity)        AS quantity,
    order_date,                    -- already TIMESTAMP
    COALESCE(discount_pct, 0)      AS discount_pct,
    CURRENT_TIMESTAMP()            AS processed_ts
FROM retail.bronze.raw_sales
WHERE TRY_TO_NUMBER(quantity) IS NOT NULL;


CREATE OR REPLACE TABLE retail.silver.stores AS
SELECT
    store_id,
    TRIM(store_name)      AS store_name,
    CURRENT_TIMESTAMP()   AS processed_ts
FROM retail.bronze.stores
WHERE store_id IS NOT NULL;
```

Gold_layer:

```
--
-------------------------gold Layer-----------------------------------------

CREATE OR REPLACE TABLE retail.gold.product_dim AS
SELECT
    ROW_NUMBER() OVER (ORDER BY product_id) AS product_key,
    product_id,
    product_name,
    category,
    price
FROM retail.silver.products;

CREATE OR REPLACE TABLE retail.gold.store_dim AS
SELECT
    ROW_NUMBER() OVER (ORDER BY store_id) AS store_key,
    store_id,
    store_name
FROM retail.silver.stores;

CREATE OR REPLACE TABLE retail.gold.customer_dim AS
SELECT
    ROW_NUMBER() OVER (ORDER BY customer_id) AS customer_key,
    customer_id,
    customer_name,
    gender,
    age,
    city
FROM retail.silver.customers;

CREATE OR REPLACE TABLE retail.gold.date_dim AS
SELECT DISTINCT
    order_date                      AS date_key,
    YEAR(order_date)                AS year,
    MONTH(order_date)               AS month,
    DAY(order_date)                 AS day
FROM retail.silver.sales;
```

```sql

CREATE OR REPLACE TABLE retail.gold.date_dim AS
SELECT DISTINCT
    order_date                      AS date_key,
    YEAR(order_date)                AS year,
    MONTH(order_date)               AS month,
    DAY(order_date)                 AS day
FROM retail.silver.sales;

CREATE OR REPLACE TABLE retail.gold.sales_fact AS
SELECT
    st.store_key,
    pr.product_key,
    cu.customer_key,
    dt.date_key,
    COALESCE(sl.quantity, 0)      AS quantity,
    COALESCE(sl.discount_pct, 0)  AS discount_pct
FROM retail.gold.store_dim st
LEFT JOIN retail.silver.sales sl
    ON st.store_id = sl.store_id
LEFT JOIN retail.gold.product_dim pr
    ON sl.product_id = pr.product_id
LEFT JOIN retail.gold.customer_dim cu
    ON sl.customer_id = cu.customer_id
LEFT JOIN retail.gold.date_dim dt
    ON DATE(sl.order_date) = dt.date_key;
```

Kpi:

```sql

CREATE OR REPLACE DYNAMIC TABLE retail.gold.dt_total_sales
WAREHOUSE = COMPUTE_WH
TARGET_LAG = '5 minutes'
AS
SELECT
    COUNT(*)        AS order_volume,
    SUM(quantity)   AS total_sales_quantity
FROM retail.gold.sales_fact;

SELECT * FROM retail.gold.dt_total_sales;


CREATE OR REPLACE DYNAMIC TABLE retail.gold.dt_top_products
WAREHOUSE = COMPUTE_WH
TARGET_LAG = '5 minutes'
AS
SELECT
    p.product_name,
    p.category,
    SUM(f.quantity) AS total_quantity
FROM retail.gold.sales_fact f
JOIN retail.gold.product_dim p
    ON f.product_key = p.product_key
GROUP BY p.product_name, p.category;

SELECT
    product_name,
    category,
    total_quantity
FROM retail.gold.dt_top_products
ORDER BY total_quantity DESC
limit 10;
```

```sql
CREATE OR REPLACE DYNAMIC TABLE retail.gold.dt_customer_behavior
WAREHOUSE = COMPUTE_WH
TARGET_LAG = '5 minutes'
AS
SELECT
    c.customer_id,
    COUNT(*)        AS total_orders,
    SUM(f.quantity) AS total_quantity
FROM retail.gold.sales_fact f
JOIN retail.gold.customer_dim c
    ON f.customer_key = c.customer_key
GROUP BY c.customer_id;

SELECT
    customer_id,
    total_orders
FROM retail.gold.dt_customer_behavior
WHERE total_orders > 1;


CREATE OR REPLACE DYNAMIC TABLE retail.gold.dt_sales_trends
WAREHOUSE = COMPUTE_WH
TARGET_LAG = '5 minutes'
AS
SELECT
    d.year,
    d.month,
    COUNT(*)        AS order_volume,
    SUM(f.quantity) AS total_quantity,
    AVG(f.quantity) AS avg_order_value
FROM retail.gold.sales_fact f
JOIN retail.gold.date_dim d
    ON f.date_key = d.date_key
GROUP BY d.year, d.month;
```

```
---
SELECT
    year,
    month,
    order_volume,
    total_quantity,
    avg_order_value
FROM retail.gold.dt_sales_trends
ORDER BY year, month;



CREATE OR REPLACE DYNAMIC TABLE retail.gold.dt_store_performance
WAREHOUSE = COMPUTE_WH
TARGET_LAG = '5 minutes'
AS
SELECT
    s.store_name,
    SUM(f.quantity) AS total_quantity
FROM retail.gold.sales_fact f
JOIN retail.gold.store_dim s
    ON f.store_key = s.store_key
GROUP BY s.store_name;

SELECT
    customer_id,
    total_orders
FROM retail.gold.dt_customer_behavior
WHERE total_orders > 1;

```